The Activity Selection Problem: Greedy, Proven Correct

Programming and Data Structures — Week 10, Lecture 1

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state the activity selection problem precisely; implement the greedy "earliest finish time" strategy and trace it completely on a concrete example; state the greedy-choice property and prove, using a formal exchange argument, that it holds for this specific problem; and explain precisely why a proof is required here, rather than a strategy that merely sounds plausible, directly connecting back to Week 9 Lecture 4's coin-change counterexample.

Outline

Today covers a single-room meeting-scheduler analogy; the activity selection problem, stated precisely; three plausible-sounding greedy strategies, only one of which is actually correct; the correct "earliest finish time" strategy, traced completely; the greedy-choice property defined precisely; the exchange argument proving earliest- finish-time correct; the MTech-track material showing concrete counterexamples for the two incorrect strategies; an in-class quiz; and a summary.

The single-room meeting-scheduler analogy

Picture a single conference room, with many people each requesting a specific, fixed time slot — a start time and an end time. The goal is to accept the maximum possible number of non-overlapping meetings, not necessarily the meetings with the highest total duration, and not merely to minimize rejections in some other sense — specifically the largest possible count of accepted, mutually compatible meetings. Week 9 Lecture 4 demonstrated that a plausible-sounding greedy strategy can be outright wrong, using coin change with denominations $[1, 3, 4]$ as a concrete counterexample. This lecture asks the natural follow-up question: is there a strategy for this new problem that is not merely plausible, but actually, provably correct?

The activity selection problem, stated precisely

The activity selection problem takes as input n activities, each defined by a start time s_i and a finish time f_i. Two activities are compatible if they do not overlap in time — activity j starts at or after activity i finishes, or vice versa. The output required is the largest possible subset of activities that are pairwise mutually compatible — no two selected activities may overlap with each other.

Three plausible-sounding strategies

Three strategies all sound reasonable at first glance: always picking the activity of shortest duration; always picking the activity that conflicts with the fewest others; or always picking the activity that finishes earliest. Each has an intuitive appeal — shorter activities seem to "waste less time," fewer conflicts seems to preserve more future options, and finishing earliest seems to leave the most room for what follows. Only one of these three is actually correct, and this lecture's MTech addition constructs concrete counterexamples defeating the other two — precisely Week 9's central lesson, that intuitive plausibility is never a substitute for proof.

The correct strategy: earliest finish time first

```
def activity_selection(activities):
    # activities: list of (start, finish) tuples
    activities = sorted(activities, key=lambda a: a[1])   # sort by FINISH time
    selected = [activities[0]]
    last_finish = activities[0][1]
    for start, finish in activities[1:]:
        if start >= last_finish:            # compatible with everything selected so far
            selected.append((start, finish))
            last_finish = finish
    return selected
```

The correct algorithm sorts activities once by finish time, an $O(n \log n)$ operation directly reusing Week 6's sorting algorithms, and then performs a single linear scan, greedily accepting any activity whose start time is at or after the most recently accepted activity's finish time. Crucially, there is no backtracking and no reconsideration of earlier decisions — each acceptance or rejection, once made, is final, which is the defining behavioral signature of a greedy algorithm, as distinct from Week 9's dynamic programming, which explicitly considers and compares multiple options before committing.

Tracing activity selection completely

```
Activities (start, finish): (1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11), (8,12)
Sorted by finish: (1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11), (8,12), (2,14),

Select (1,4). last_finish=4
(3,5): 3<4, skip
(0,6): 0<4, skip
(5,7): 5>=4, SELECT. last_finish=7
(3,9): 3<7, skip
(5,9): 5<7, skip
(6,10): 6<7, skip
(8,11): 8>=7, SELECT. last_finish=11
(8,12): 8<11, skip
(2,14): 2<11, skip
(12,16): 12>=11, SELECT. last_finish=16

Result: (1,4), (5,7), (8,11), (12,16) - 4 activities
```

Tracing this well-known 11-activity instance completely: after sorting by finish time, the scan accepts $(1, 4)$ first, then skips every activity starting before time 4, accepting $(5, 7)$ next. It then skips every activity starting before time 7, accepting $(8, 11)$, and finally skips activities starting before 11, accepting $(12, 16)$. The result, 4 mutually compatible activities, can be checked directly: no two selected intervals overlap, and this is provably the maximum possible count for this specific instance — a claim the next two slides prove rigorously, not merely assert.

The greedy-choice property, defined precisely

A problem exhibits the greedy-choice property if making a locally optimal choice at each step, and never reconsidering that choice afterward, still leads to a globally optimal overall solution. This is a substantially stronger requirement than Week 9's optimal substructure alone — many problems have optimal substructure (an optimal solution can be built from optimal solutions to subproblems) but lack the greedy- choice property specifically, meaning the subproblems must still be solved exhaustively via dynamic programming rather than committed to greedily. Coin change with denominations $[1, 3, 4]$ is exactly such a case: it has optimal substructure (Week 9's DP solution relies on this), but greedy's "always take the largest coin" choice lacks the greedy- choice property, since Week 9 Lecture 4 proved that choice does not always lead to the global optimum.

The exchange argument: proving earliest-finish-time correct

The exchange argument proves the claim that some optimal solution always includes the activity with the earliest finish time, a_1. Let O be any optimal solution to the problem. If a_1 is already in O, the claim holds trivially. If not, let a_k denote whichever activity in O happens to have the earliest finish time among O's members. Since a_1 has the earliest finish time among *all* activities, in particular $f_1 \leq f_k$. Replacing a_k with a_1 inside O cannot introduce any new overlap: every other activity in O was already compatible with a_k (meaning it started no earlier than f_k), and since $f_1 \leq f_k$, it is automatically compatible with a_1 too. The modified set is therefore still valid, and has exactly the same size as O, so it is also optimal — proving an optimal solution containing a_1 always exists.

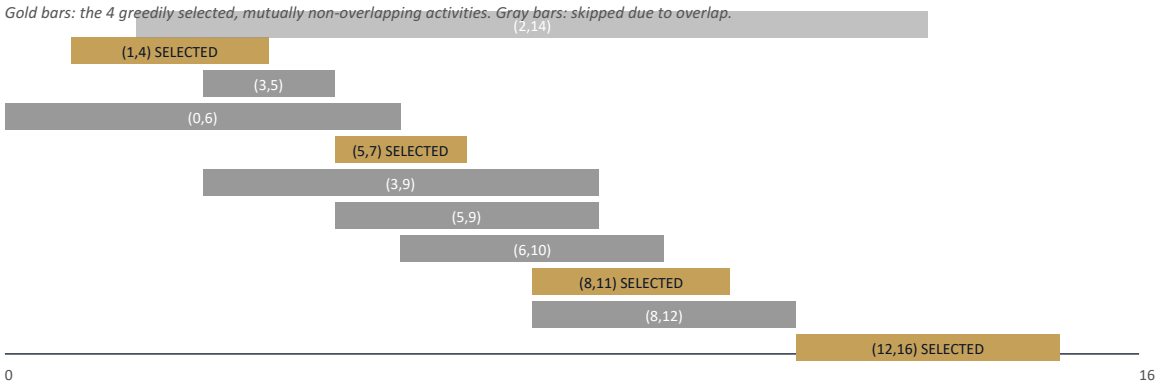
Why the timeline analogy helps make the exchange argument concrete

It helps to picture the exchange argument physically: draw every activity as a horizontal bar on a timeline. The bar for a_1 ends at or before the point where every other candidate activity's bar ends, since a_1 has the globally earliest finish time. "Sliding" a_k's bar leftward until it aligns with where a_1's bar sits cannot create any new collision with activities to its right, precisely because a_1's bar ends no later than a_k's did — anything compatible with the later endpoint remains compatible with the earlier one. This physical picture is exactly what the algebraic inequality $f_1 \leq f_k$ encodes formally, and is often the more intuitive way to internalize why the exchange argument works.

Completing the proof by induction

The exchange argument shows that after greedily selecting a_1, the remaining problem — finding the maximum compatible set among activities starting no earlier than f_1 — is a strictly smaller instance of the exact same activity selection problem. By induction on the number of remaining activities, the identical exchange argument justifies the greedy choice at every subsequent step, all the way down to the base case of zero remaining activities. This completes a full correctness proof: it relies on optimal substructure, established via induction on smaller instances of the same problem, combined with the greedy-choice property, established via the exchange argument — both genuinely required together, not merely one alone.

The picture: the greedy scan, visualized as a timeline



Seeing all eleven activities laid out on a shared timeline makes the greedy scan's behavior visually immediate: each selected activity begins only after the previously selected one has genuinely finished, and every skipped activity visibly overlaps with an already-selected one.

MTech addition — why "shortest duration first" fails, verified

```
Activities: (0,5), (4,6), (5,10)

Shortest-duration-first: durations are 5, 2, 5 → pick (4,6) first (shortest)
  (4,6) conflicts with BOTH (0,5) [5>4] and (5,10) [5<6] → nothing else fits
  Shortest-first result: {(4,6)} - only 1 activity

True optimum (brute force, checked): {(0,5), (5,10)} - 2 activities, fully compatible
```

This is a genuine, brute-force-verified counterexample, not a hypothetical concern. With activities $(0, 5)$, $(4, 6)$, and $(5, 10)$, shortest-duration-first selects $(4, 6)$ first, since its duration (2) is the smallest. But $(4, 6)$ overlaps both $(0, 5)$ and $(5, 10)$, which are themselves mutually compatible with each other — so this one greedy choice blocks a selection of 2 activities, leaving shortest-first with only 1. Exhaustive brute-force checking confirms the true optimum for this instance is 2, achieved by $\{(0, 5), (5, 10)\}$. This directly mirrors Week 9's coin-change lesson: a locally appealing metric (short duration) does not track genuine global optimality, and only earliest- finish-time survives the exchange-argument proof given earlier in this lecture. A similar construction defeats fewest-conflicts-first, left as this lecture's lab exercise.

Exercise

As an exercise, implement `activity_selection` exactly as presented in this lecture, and verify it against the 11-activity example traced here, confirming it selects the identical 4 activities. Separately, extend the exchange-argument proof in writing to explicitly handle the case where two activities share the identical finish time, addressing whether the specific tie-breaking rule used when sorting matters for the algorithm's overall correctness.

Cost of the greedy algorithm

The complete algorithm costs $O(n \log n)$ for the initial sort by finish time, directly reusing Week 6's sorting algorithms, plus $O(n)$ for the single subsequent linear scan — dominated entirely by the sort, giving $O(n \log n)$ total. This is dramatically cheaper than a dynamic programming approach would need for a comparably structured optimization problem, which typically requires examining a table of subproblems rather than a single linear pass. This speed is the concrete reward for having proven the greedy-choice property genuinely holds: a correct (if unnecessary) dynamic programming solution to activity selection also exists, but this problem's stronger structural property makes that extra machinery unneeded here.

Quiz — apply the strategy

Given activities with start and finish times $(1, 3)$, $(2, 5)$, $(4, 6)$, and $(6, 8)$, apply the earliest-finish-time-first strategy from this lecture and determine which activities are selected, before checking the answer.

A second worked trace, with a tie

Tracing a second example specifically constructed to include a tie: $(1, 4)$ and $(2, 4)$ share the identical finish time, 4. Breaking the tie by start time here selects $(1, 4)$ first; the algorithm then correctly skips $(2, 4)$, since it starts before time 4, and selects $(4, 6)$, giving 2 total activities. Checking the alternative tie-break — selecting $(2, 4)$ first instead — produces the identical final count of 2, confirming the exercise's earlier question: the specific tie-breaking rule does not affect correctness here, only which of several equally optimal solutions is returned.

Quiz — answer

Sorted by finish time, these activities are already in order: $(1, 3), (2, 5), (4, 6), (6, 8)$. The first, $(1, 3)$, is selected. $(2, 5)$ starts at 2, before the accepted activity's finish at 3, so it is skipped. $(4, 6)$ starts at 4, at or after 3, so it is selected, updating the last finish time to 6. $(6, 8)$ starts at 6, at or after 6, so it is

also selected. The final result is $(1, 3), (4, 6), (6, 8)$ — 3 activities, which can be checked directly to be the true maximum for this instance.

A closing connection to Week 8's priority queue

As a closing connection: if activities are not all known upfront but arrive dynamically over time, a min-heap keyed by finish time, directly reusing Week 8's structure, replaces the one-time sort this lecture used. Each newly arriving activity is inserted in $O(\log n)$, and the next candidate to evaluate is always available via `peek()` in O(1), without needing to re-sort the entire remaining set. The underlying greedy logic — always evaluate compatibility against the earliest- finishing candidate — is unchanged; only the supporting data structure adapts to the streaming setting, a useful preview of how this unit's ideas extend beyond the static, all-at-once problem statement given here.

Summary

Activity selection asks for the maximum number of pairwise non- overlapping activities, and earliest-finish-time-first is the strategy that is actually, provably correct — the other two plausible-sounding strategies are not, as this lecture's MTech addition demonstrates concretely. The greedy-choice property, established here via a formal exchange argument, is a substantially stronger requirement than Week 9's optimal substructure alone, and many problems possessing optimal substructure still lack it, exactly as coin change with $[1, 3, 4]$ demonstrated. Every greedy algorithm this course endorses from this point forward must have this property genuinely proven for that specific problem, never merely assumed by loose analogy to a previously correct case. Next lecture introduces Huffman coding, a second genuinely proven-correct greedy algorithm, requiring its own distinct exchange argument.